University of Staffordshire | A·P·I·I·T Inspire love for learning

INDIVIDUAL ASSIGNMENT

LEVEL 05

COMP50056

Advanced Machine Learning

AI2521

Hand Out Date: 27-Jul-26

Hand In Date: 5-Oct-26

Student NO: CB016537

Table of Contents

1. Introduction and Scenario 3
2. The Business Problem 3
3. Data Preparation and Preprocessing 3
3.1 Augmentation and Class Balancing 4
4. Model Design and Architecture 5
4.1 Model A — CNN Trained from Scratch 5
4.2 Model B — MobileNetV2 Transfer Learning 5
5. Hyperparameters and Experimental Protocol 5
5.1 Tuning the Learning Rate 6
6. Results and Evaluation 6
6.1 Deployment Cost 9
7. Limitations 9
8. Ethical and Privacy Considerations 9
9. Business Recommendations 10
10. Conclusion 10
References 11

1. Introduction and Scenario

SmartSense AI is working on HomeVision, an indoor monitoring solution for residential homes and smart buildings. Means of surveillance must recognize the area being viewed in order for the building to regulate illumination and ventilation, manage the energy consumption, and carry out contextually aware automation without being prompted. The prototype product encounters difficulties because actual interiors are visually inconsistent, ranging from several sources of light to the single lighting device, with furniture being moved around the area and cameras installed at angles that appear odd.

The set of images used in the process is called MIT Indoor Scenes (Quattoni and Torralba, 2009), which represents an image corpus comprising 15,620 images across 67 types of interiors. Out of all the interior scenes, I picked five, including bedroom, kitchen, living room, restaurant, and bar (a total of 3,219 images). The key restriction was that not less than 400 images for each interior category had to be available in order to have a lesser extent of duplication. The second premise of choosing the categories is the requirement of their similarity with each other: restaurant and bar share seating, tables, and lighting as the distinction is affected by cues that happened to be either not visible or poorly captured

2. The Business Problem

If a system is not aware of whether it is viewing a kitchen or a bedroom, it will not be able to utilize the extraction profile, the sleep light scheme, or regulations on occupancy management. Ultimately, all of the devices the company is selling depend on this inference.The solution is limited by three business factors. The costs of mistakes are uneven: cutting the lights in an incorrect room is insignificant, while activating the ventilation system incurs great expenses and creates mistrust, which suggests that the system has to have a high level of certainty instead of simple prediction. The inference must be executed on inexpensive devices since streaming video increases the cost and puts privacy at risk. And finally, the device can be sold only in Sri Lanka, while the data used to train the system come mostly from the west and are filmed professionally, which creates impossible distribution obstacles.

3. Data Preparation and Preprocessing

Due to lacking rare classes in the 400-image floor, the resulting imbalance is minimal, whereby kitchen has a 734:513 and restaurant a 1.43:1. Consequently, stratified splitting of 70/15/15 yields good results: with 2253 images, the training set, 483 the validation and 483 the test set, and with the operation being perform at the file level confirming that images are not duplicated across partitions, while the test set being a single use.

The integrity pass processes and rejects the files before modelling that are either truncated or non-RGB. These files do not raise any errors upon download but fail during the trains loop that makes it very hard to debug during an epoch.

The images are resized to 224x224, whereby resizing is better than centre-cropping as often the data defining a space is off-centre. The normalisation happens within each model in such a way that in

the case of a from-scratch network range [0, 1], the MobileNetV2 preprocess_input is applied. Error is not raised, as the regularity of a backbone relying on an incorrect input range causes the accuracy loss without a sign of error (Chollet, 2021): hence binding each model to its own scaling eliminates such a mistake.

3.1 Augmentation and Class Balancing

Every augmentation category simulates a defined failure condition; the information can be seen in Table 1. However, I have abandoned the operations that include vertical flips and excessive rotation. A camera mounted on the ceiling is incapable of capturing upside-down photographs of the room and the results will bring unnecessary expenses for the process of augmenting images which would never be used (Shorten and Khoshgoftaar, 2019). The layers are already implemented in the model, therefore, Keras will work with them even during the training phase. The following part explains how much the ablation has contributed to further processing.

Deployment problem	Augmentation	Justification
Variable lighting	RandomBrightness / RandomContrast ±20%	A room at noon, at dusk and under one lamp must map to the same class.
Camera angle and tilt	RandomRotation ±10%, horizontal flip	Mounts are rarely level; layouts are frequently mirrored.
Distance and framing	RandomZoom ±15%	Mounting distance and focal length vary between installations.
Clutter and object position	RandomTranslation ±10%	Discourages reliance on the diagnostic object sitting centrally.

Table 1: Augmentation choices mapped to stated deployment conditions.



Figure 1: One training image under twelve random augmentations. Every variant is one a fixed ceiling camera could plausibly produce.

Next, I resampled the whole training set and secured 500 pictures per class, which means I worked with 2,500 images in total. In the process, I had to down-sample the kitchen images and up-sample the other groups to secure 500 images per class. I resampled the training dataset only after splitting the data, and as a result, I managed to prevent duplicates from showing up in validation test results. I also kept everything from original samples during the resampling process.

4. Model Design and Architecture

Two models were implemented, paired so the comparison isolates a single variable: the presence of ImageNet pre-training.

4.1 Model A — CNN Trained from Scratch

The baseline consists of a VGG-like architecture consisting of four blocks, during which the number of filters increases from 32 to 256, with each block performing two 3x3 convolutions with batch normalization and ReLU followed by a max pooling layer. In this way, two sets of 3x3 convolutions achieve the same field of vision as a 5x5 kernel but with far fewer parameters and with added non-linearity (Simonyan and Zisserman, 2015). The significance of batch normalization lies in the fact that it helps make the training more stable and provides a certain degree of regularization (Ioffe and Szegedy, 2015).

The head of the network utilizes global average pooling instead of flattening the data. The flattening of the obtained 14×14×256 map into a fully connected layer would require around 6.4 million parameters, which is even more than the overall number of parameters in the convolutional blocks and is the place of maximum overfitting of the small data network. Afterwards dropout with the probability of 0.3 is implemented before the softmax classifier that consists of 5 units (Srivastava et al., 2014).

4.2 Model B — MobileNetV2 Transfer Learning

The second model adheres to the conventional protocol, which entails removing the ImageNet classifier from MobileNetV2's architecture, making the required preprocessing using the backbone, applying a global average pooling layer, dropout, and softmax classifier with five outputs, and then fine-tuning the model at a learning rate of 1e-5 after unblocking the last 30 layers of the architecture.

The reason for employing the method is that the data volume is at odds with the similarity between the domains. The total amount of training data is approximately 2,250 images, which are quite similar to the samples from ImageNet, so feature extraction followed by minimal fine-tuning is a successful approach (Yosinski et al., 2014). The model has already established representations of edges and shapes (bed, sink, table), thus the only remaining goal is to dictate the mapping from parts to categories.

I choose MobileNetV2 instead of larger backbones during deployment because depthwise separable convolutions divide convolution into channel-wise spatial filter and a 1x1 mixing computation, which reduces the number of multi-accumulation operations by a factor of roughly 8 while keeping accuracy at a satisfactory level (Sandler et al., 2018). There are two notes to be made. The frozen base is invoked with training=False so its batch-normalisation layers stay in inference mode; letting them update from batches of 32 would corrupt statistics estimated over 1.4 million images. Fine-tuning runs only after the head converges.

5. Hyperparameters and Experimental Protocol

A performance difference is interpretable only if every other condition is held constant. Table 2 records the shared protocol.

Setting	Value	Rationale
Optimiser	Adam	Adaptive per-parameter rates; removes hand-tuned momentum as a confound.
Loss	Sparse categorical cross-entropy	Integer labels, so the loss matches the label format directly.
Input size	224 × 224	MobileNetV2's native resolution, applied to both models.
Stage 1	≤30 epochs at 1e-3	Trains the head, or the whole from-scratch network, to convergence.
Stage 2	≤15 epochs at 1e-5 (A: 1e-4)	Fine-tuning; a normal rate would overwrite pre-trained weights.
Total budget	≤45 epochs for both	Neither model gets a longer race than the other.
Early stopping	val_loss, patience 6	The budget is a ceiling, not a mandate.
Training balance	500 images/class	Resampled after the split, training set only; max 1.5× duplication.

Table 2: Training protocol, held identical across both models.

Model A fine-tunes at 1e-4 rather than 1e-5: fairness requires an equal budget and stopping rule, not an identical number meaning different things in each setting, and Model A has no pre-trained weights to protect. Early stopping restores the best weights only within its own stage, so a checkpoint spanning both stages is reloaded before evaluation.

5.1 Tuning the Learning Rate

Two hyperparameters are adjusted automatically instead of set manually: early stopping determines how many epochs should be done, and ReduceLROnPlateau decreases the learning rate when the validation error does not improve anymore. Three models were trained using the same frozen-base model and from the same seed on a limited budget and early stopping was disabled. The achieving 1e-3 demonstrated the best validation accuracy value of 0.8178 followed by 1e-2 with 0.7867 and 1e-4 with 0.7164 representing the difference of 0.031 in relation to the second-best value. The budget frame does impact on 1e-4 as this rate still has a chance for improvement when it came to the end of training, thus, this not a proof of any failure of the strategy.

6. Results and Evaluation

Macro-averaged F1 is the primary metric: it weights every class equally, reflecting that a space the system never recognises is a broken feature for the building that has it, however rare it is in training.

Model	Accuracy	Macro F1	Weighted F1	Top-2 acc.	Epochs
CNN from scratch	0.6522	0.6472	0.6479	0.8696	45
MobileNetV2 (transfer)	0.8012	0.7986	0.7976	0.9524	28

The results of using transfer learning were getting a macro F1 score higher by 0.151, which means 23.4% increase, in the course of training that lasted 28 epochs and took about 3.4 minutes as opposed to working 45 epochs and spending 18.6 minutes that represents fivefold shorter time for training and increased accuracy. With 0.652 accuracy, my from-scratch network is a credible baseline rather than a straw man, so the gap reflects what pre-training supplies. Fine-tuning added a further 0.008 validation accuracy (0.814 to 0.822), confirming the frozen features were already close to sufficient.

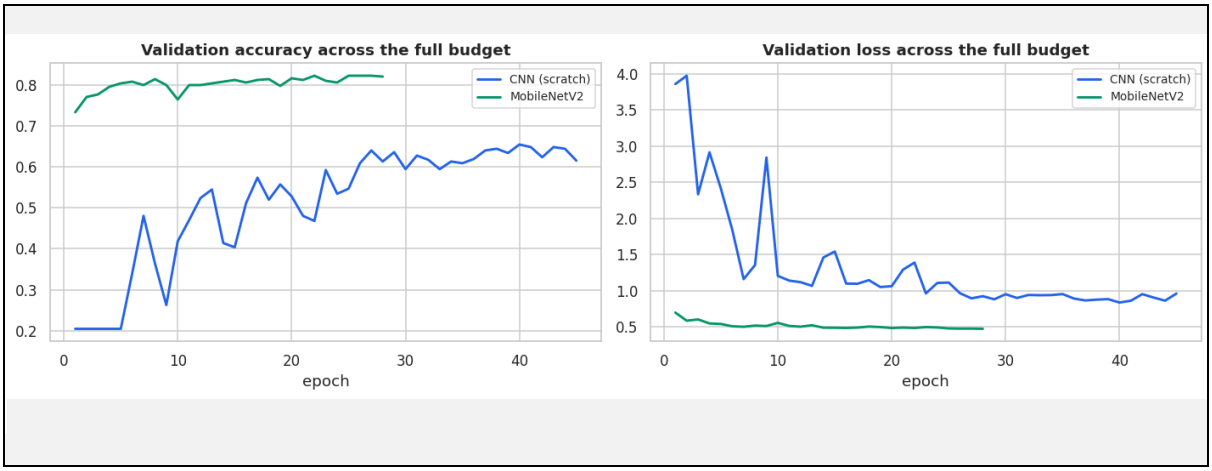


Figure 2: Validation accuracy and loss for both models across the full budget.

The observation of top-2 accuracy of 0.9524 compared to the top-1 of 0.8012 provides the key insight that in approx. 15% of the cases the correct class is not picked first but the second implying that the features used are correct and the error belongs to the decision boundary instead of the misclassification decision. The confusion matrix reveals the location of the decision boundary which is surprisingly not where I thought it would be. I picked restaurant and bar as the purposely difficult pair and they do confuse each other (restaurants misclassified as bars at 14% and vice versa at 9%) but nevertheless both succeed well with F1 at 0.80 and 0.85 respectively. The weakest class here is livingroom with recall of 0.60 and F1 of 0.68 whose main reason for failure is getting identified as bedroom: 20% of livingrooms were classified like that which is the biggest single off-diagonal cell in the matrix. The prediction was wrong, but the method that exposed it was not: per-class metrics and a confusion matrix surfaced a weakness that overall accuracy would have hidden.

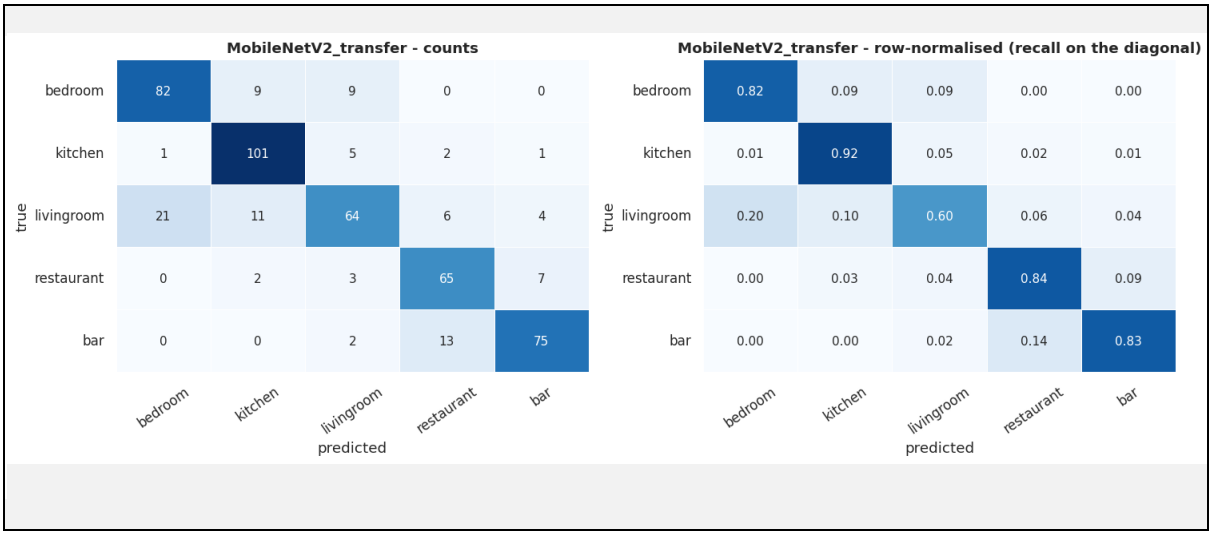


Figure 3: Row-normalised confusion matrix for MobileNetV2; the diagonal shows per-class recall.

Per-class figures matter more than the headline here, because a single average can conceal a class the system never recognises. Figure 4 reports precision, recall and F1 for each of the five classes against the macro average.

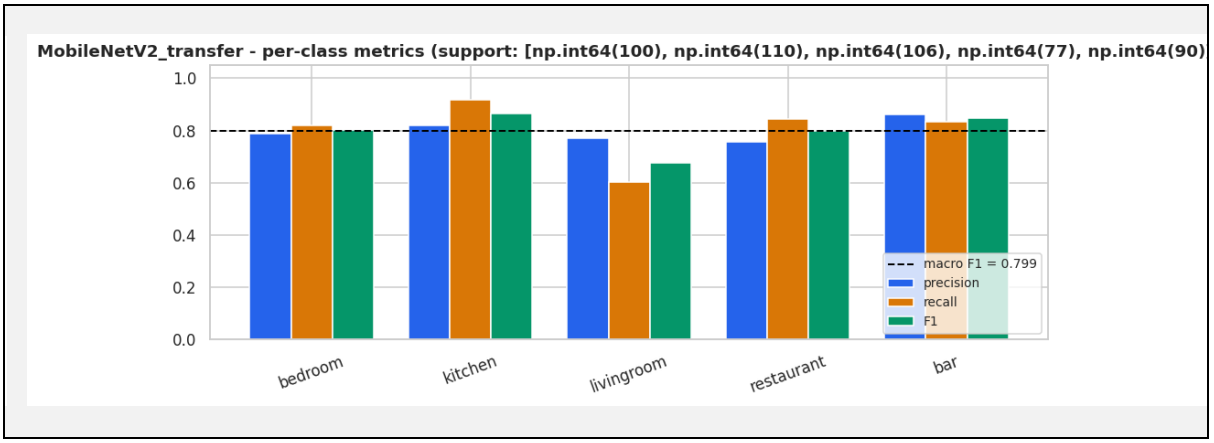


Figure 4: Per-class precision, recall and F1, with the macro average marked.

A confidence-threshold sweep withheld low-confidence predictions; accuracy rises materially as coverage falls, showing confidence is informative about correctness — the property the abstention policy in Section 9 requires. Grad-CAM (Selvaraju et al., 2017) confirmed predictions rest on diagnostic objects rather than background.

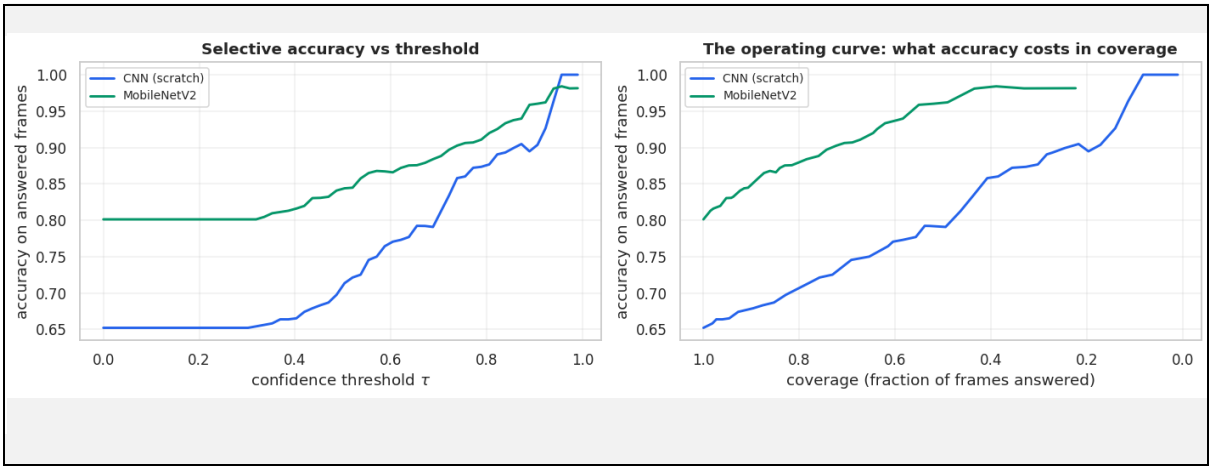


Figure 5: Accuracy on answered frames as coverage falls. A steep rise means confidence is informative, which is what makes abstention viable.

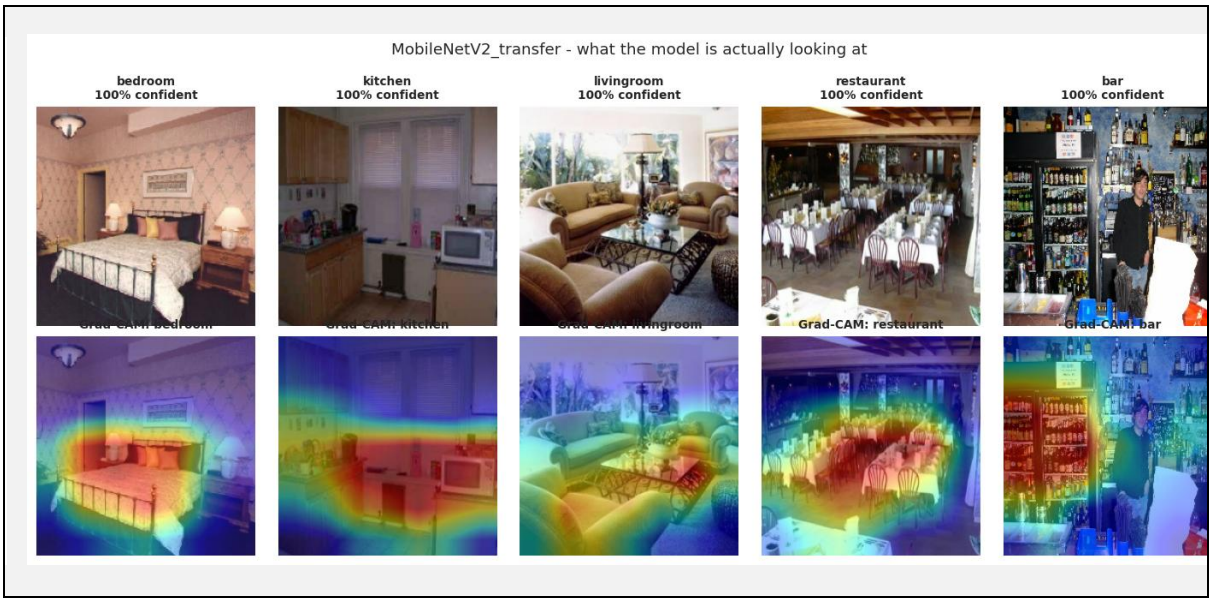


Figure 6: Grad-CAM overlays. Heat concentrated on functionally diagnostic objects indicates the model learned the space's function rather than an incidental correlate.

6.1 Deployment Cost

Model	Params (M)	TFLite int8 (MB)	Latency (ms/image)
CNN from scratch	1.209	1.234	27.3
MobileNetV2 (transfer)	2.264	2.512	149.9

Table 4: Exported model size and single-image latency, measured on a Colab T4 GPU.

This finding does not seem intuitive, being worth articulating concisely; MobileNetV2 is approximately five times slower for each image as compared with the smaller architecture, even though being an efficient design. The reason is not the model itself but rather the way measurement was conducted. MobileNetV2 was optimized for multiply-accumulate operations rather than throughput; its depthwise convolution operations are constrained by memory bandwidth, and thus they do not completely utilize T4. The conclusions drawn about the two networks will be different when using the CPU or NPU hardware the application is supposed to work with, which means that multiple quantitative comparisons need to be made before coming to the final installation decision.

7. Limitations

The biggest shortcoming can be seen in domain transfer. While images from the MIT dataset represent photographs taken by humans that are stable in lighting and composition, images taken by the ceiling camera will show wide-angle and poorly lit scenes filled with people and clutter. So, the accuracy that I reported can be considered only an optimistic upper bound, so this result must be verified on video data before making commercial claims.

Moreover, there exist 4 additional limitations. First of all, it should be emphasized that the classifier is based on a closed set of classes, which means that hallways, balconies, and offices will be misclassified with a lot of confidence. Also, having analyzed only one 483-image test dataset, I am able to receive just a point estimate, but the idea of k-fold cross-validation is planned for future work. Besides, images are analyzed independently, which means that the best prior information is ignored, namely that a fixed camera is able to see the same space from its previous position with a very small time lag. Finally, the data is predominantly Western and skewed towards affluent, tidy interiors while the product sells into Sri Lanka, so accuracy is likely lowest in precisely the buildings the company intends to serve: a fairness concern, not merely an accuracy one.

8. Ethical and Privacy Considerations

Indoor cameras are among the most intrusive consumer sensors in existence, and bedroom is one of the five classes. The hospitality classes raise a distinct concern: people filmed in a restaurant or bar are customers who have agreed to nothing. The following are design requirements, not aspirations.

- On-device processing. Classification should run inside the camera so that only a label and confidence value leave the device, never an image. This is simultaneously the strongest privacy guarantee, the cheapest option and the lowest-latency one.

- No retention by default. Frames should exist in a rolling buffer and be overwritten; any training-data collection must be separately consented and revocable.
- Hard exclusion zones. Bedrooms warrant a firmware-level camera disable that software cannot override.
- Lawful basis and minimisation. Indoor imagery of identifiable premises is personal data; purpose and storage limitation are obligations under UK and EU GDPR, with comparable duties under Sri Lanka's Personal Data Protection Act No. 9 of 2022.
- Function creep. A space classifier is one modification away from an occupancy tracker, and occupancy data reveals routines and absences. Contractual and architectural limits on secondary use should be set now, while the system is still a prototype.

Performance should additionally be reported per class and per deployment segment rather than as a single global figure, since degradation for smaller or more cluttered premises would otherwise remain invisible in the headline number — the same principle that motivates macro-averaged F1 at class level.

9. Business Recommendations

- Deploy the transfer-learned model. The same engineering budget buys 0.151 more macro F1 and a 5.5-fold shorter training cycle simply by not starting from zero.
- Re-measure latency on the target device before fixing the deployment architecture. Table 4 ranks the models on a GPU the product will never use, and that ranking is likely to invert on edge hardware.
- Operate with a confidence threshold and an explicit unknown state, tied to the cost of the action: low for reversible changes such as dimming, high for anything expensive or irreversible. Taking no action is a legitimate automation decision.
- Implement temporal smoothing before investing in model capacity. A rolling vote across consecutive frames would remove most single-frame instability at negligible cost and without retraining, making it the highest value-per-effort improvement available.
- Fund a Sri Lankan pilot dataset as a launch dependency. Without it the quoted accuracy cannot honestly be claimed for the target market.

A sixth option merits consideration, though the evidence redirects it. I had expected restaurant and bar to be the candidates for merging; the measured weakness is livingroom against bedroom, with 20% of livingrooms misread. Merging those two is not viable, since they drive opposite automation: sleep-aware lighting and night-time HVAC setback in one, shared-occupancy behaviour in the other. The practical response is to collect more livingroom and bedroom examples, particularly cluttered and partially framed ones, and to lean on the temporal smoothing in recommendation four, which would resolve most of this confusion for a fixed camera.

10. Conclusion

For HomeVision, I have compared two distinct deep learning methodologies for indoor scene recognition. The technology of the mobile architecture developed for that purpose, MobileNet V2, resulted in significant improvement in performance over the conventional model, with the macro F1

score value of 0.7986, while the macro F1 score value for the model that was developed and trained from scratch is only 0.6472, which indicates that the technology of MobileNet has about 23.4 percent competitive advantage, while the training period has only been about one-fifth of the conventional alternative. Furthermore, it turned out that the model we created for restaurants works very well, while the performance of the model in living room scenes is the worst with the lowest value of recall which is only equal to 0.6, with most of confusion being about living room and bedroom recognition. The claims of deployment are restricted due to three limitations: the model has not been tested on real footage coming from the cameras of the target audience, the latency numbers were received using the hardware that it will not use for its applications, and the livingroom weakness needs addressing before a room-level automation rule can be trusted.

References

Chollet, F. (2021) Deep Learning with Python. 2nd edn. Shelter Island, NY: Manning Publications.

Ioffe, S. and Szegedy, C. (2015) 'Batch normalization: accelerating deep network training by reducing internal covariate shift', Proceedings of the 32nd International Conference on Machine Learning, pp. 448-456.

Kingma, D.P. and Ba, J. (2015) 'Adam: a method for stochastic optimization', Proceedings of the 3rd International Conference on Learning Representations.

Quattoni, A. and Torralba, A. (2009) 'Recognizing indoor scenes', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 413-420.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018) 'MobileNetV2: inverted residuals and linear bottlenecks', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4510-4520.

Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017) 'Grad-CAM: visual explanations from deep networks via gradient-based localization', Proceedings of the IEEE International Conference on Computer Vision, pp. 618-626.

Shorten, C. and Khoshgoftaar, T.M. (2019) 'A survey on image data augmentation for deep learning', Journal of Big Data, 6(60), pp. 1-48.

Simonyan, K. and Zisserman, A. (2015) 'Very deep convolutional networks for large-scale image recognition', Proceedings of the 3rd International Conference on Learning Representations.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: a simple way to prevent neural networks from overfitting', Journal of Machine Learning Research, 15(1), pp. 1929-1958.

Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014) 'How transferable are features in deep neural networks?', Advances in Neural Information Processing Systems 27, pp. 3320-3328.